DealScout Viva Companion

Use this guide to explain the system in simple, accurate language. Start with the problem, show the workflow, then discuss technology and limitations.

- One-line definition: DealScout compares normalized product offers and highlights the lowest listed price.
- Demonstration order: search → comparison → best price → login → insights.
- Technology: HTML, CSS, JavaScript (ES6+), Node.js core modules, JSON files — exactly the CS-303 Web Technology-I syllabus.

VIVA TIP

Speak clearly, demonstrate the real flow, and separate implemented behavior from planned enhancements.

60-Second Introduction

DealScout reduces the effort of visiting many stores separately. A user enters a product name, chooses a category and source mode, and receives normalized offers.

- The Node.js server validates the query.
- Offers use one common ProductOffer object structure.
- The minimum numeric price becomes the best deal.
- Each search is appended to searches.json.

VIVA TIP

Speak clearly, demonstrate the real flow, and separate implemented behavior from planned enhancements.

Problem and Objectives

The main problem is inconsistent and scattered e-commerce information.

- Save comparison time.
- Improve clarity through a common table.
- Show seller, rating, discount, and delivery with price.
- Record searches for later review.
- Run anywhere Node.js is installed.

VIVA TIP

Speak clearly, demonstrate the real flow, and separate implemented behavior from planned enhancements.

Architecture Answer

The system follows a client-server architecture with no frameworks.

- HTML, CSS and JavaScript run in the browser.
- server.js uses http.createServer to serve static files and /api routes.
- modules/*.js separate products, auth, analytics and file storage.
- JSON files in data/ store catalog, users and search logs.
- An optional https call supplies live shopping results.

VIVA TIP

Speak clearly, demonstrate the real flow, and separate implemented behavior from planned enhancements.

Search Algorithm

Explain the algorithm step by step without jargon.

- Read and trim the query.
- Reject input shorter than two characters (HTTP 400).
- Filter matching offers by query words and category.
- Normalize every accepted offer.
- Use reduce() to find the minimum price → best_deal_id.
- Append the summary to searches.json and return JSON.

VIVA TIP

Speak clearly, demonstrate the real flow, and separate implemented behavior from planned enhancements.

Node.js Modules Used

Be ready to name each core module and its purpose.

- http — create the server and handle requests/responses.
- fs (fs.promises) — read/write JSON data files and stream static files.
- path — safe file paths inside /public.
- url — URL and URLSearchParams parsing.
- crypto — randomUUID() for ids and randomBytes() for session tokens.
- https — optional live provider request.

VIVA TIP

Speak clearly, demonstrate the real flow, and separate implemented behavior from planned enhancements.

JavaScript (ES6+) Features Used

Point to concrete examples in the code.

- const/let, arrow functions, template literals.
- Destructuring and spread (...offer).
- Array methods: map, filter, reduce, find, some, flatMap.
- async/await with fetch() in the browser and fs.promises on the server.
- Map and Set, optional chaining (?.), nullish coalescing (??).

VIVA TIP

Speak clearly, demonstrate the real flow, and separate implemented behavior from planned enhancements.

Database Answer

The project stores data in JSON files, the equivalent of tables.

- products.json → sample catalog (read-only).
- users.json → login accounts.
- searches.json → search log written by a queued writer.
- Analytics are computed on request, not stored.
- Future: migrate to MongoDB (Semester V elective).

VIVA TIP

Speak clearly, demonstrate the real flow, and separate implemented behavior from planned enhancements.

Authentication Answer

The preview login creates an HTTP-only cookie session.

- POST /api/auth/login checks users.json.
- A random token is stored in a server-side Map with an 8-hour expiry.
- The cookie travels automatically with same-origin fetch() calls.
- GET /api/analytics returns 401 without a valid session.

VIVA TIP

Speak clearly, demonstrate the real flow, and separate implemented behavior from planned enhancements.

API Questions

REST endpoints exchange JSON between the browser and server.

- GET /api/products/search?q=&category=&mode= — search and compare.
- POST /api/auth/login — create session.
- GET /api/auth/me — return current user or null.
- POST /api/auth/logout — end session.
- GET /api/analytics — protected search insights.
- HTTP 400 means validation failed; 401 means login required.

VIVA TIP

Speak clearly, demonstrate the real flow, and separate implemented behavior from planned enhancements.

Live vs Sample Data

This distinction is important during the viva.

- Sample catalog is local, stable, and always available.
- Live search requires the SERPAPI_KEY environment variable on the server.
- Without a key, preview results are explicitly labelled and not claimed as live.
- Provider keys never appear in browser code.

VIVA TIP

Speak clearly, demonstrate the real flow, and separate implemented behavior from planned enhancements.

Limitations, Future Scope and Checklist

State limitations before enhancements, then prepare the environment.

- Limited local catalog; no checkout or payment; plain-text preview password.
- Future: verified providers, alerts, favorites, MongoDB, hashed passwords, admin tools.
- Before viva: run setup.bat / setup.sh, search iPhone 15, show best price, sign in, open Insights.
- Keep Word/PDF report, source folder and setup guide ready.

VIVA TIP

Speak clearly, demonstrate the real flow, and separate implemented behavior from planned enhancements.